

Linjär Algebra med fokus på 3D-matematik

Uppgift P01 - [IG/G] - Kollisioner (3/3)

Uppgiftens målsättning

Den studerande ska vidareutveckla tidigare uppgift som ger erfarenhet av att arbeta med vektorer i flera koordinatsystem med funktionalitet för att **detektera kollision** mellan enkla objekt i spel.

Uppgift

Instruktioner enligt Uppgift A02a skall följas. Från det, vidareutveckla programmet som löser uppgift A02a, så att **spelarens** rörelse är **begränsad till skeppet**. Vidare skall **två nya koncept** introduceras. Det **första** är en **stråle** (ray) som kan **skjutas från spelaren** och det **andra** är en **cirkel** som **växer över tid** och **krymper om den träffas av spelarens vapen**.

Nytt från föregående delar

Vi skall även kontrollera om **space** är nedtryckt. Om så är fallet skall en gul stråle i spelarens framåtriktning ritas. Strålen kan ritas med en **DebugDrawer.DrawLine** (på samma sätt ni ritat linjer i tidigare uppgifter), linje från spelaren med samma riktning som spelaren.

För att implementera detta behöver ni varje frame, kontrollera om **space** är **nedtryckt**, då aktiveras strålen, **utritning sker** och **kollision mot cirkel kontrolleras**. Om **inte**, så skall strålen inte ritas ut och kollisionsdetektion kontrolleras inte.

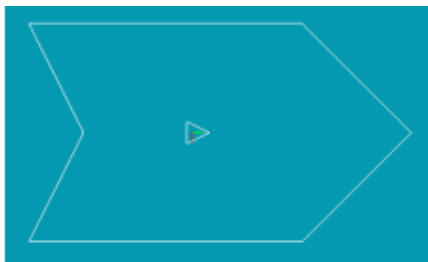
Exempel (pseudo-kod. Endast för referens, ni väljer själva hur det implementeras):

```
Player::Update()
    if (KeyPressed(space))
        auto &w = myWeaponRay;
        if ( w.IntersectsCircle ( theCircle ) )
            theCircle.Shrink();
            DebugDrawer.DrawLine(w.myFrom, w.myTo, w.myColor );
```

För skeppet

För skeppet är det **starkt** rekommenderat att se till att det är en konvex polygon, i tidigare exempel som lämnas ut ritas skeppet konkavt vilket gör testerna svårare. Så har ni följt formen på skeppet från exemplet kan ni till exempel uppdatera formen så att det är en konvex form. Eller att ni endast får röra er i en konvex version av skeppet

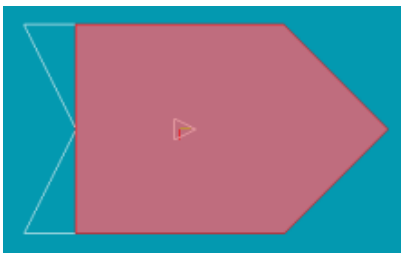
***Exempel:** om ni till exempel har det som i referensen*



Så kan ni förenkla skeppet såhär:



Eller endast testar kollision mot en förenklad version av skeppet, och inte tillåter spelaren att gå i konvexa delar:



Nytt objekt: cirkel

Rita ut en **cirkel** på en valfri eller **slumpad position**. Den skapas med **radie=0** och växer över tid. Träffas cirkeln av spelarens laser så krymper den med en faktor som är större än tillväxthastigheten, (så att den krymper). **Cirkeln behöver inte ta hänsyn till kollision med spelare eller skepp, endast spelarens vapen.** Är vi inne i cirkeln, så att vapenstrålen startar inne i cirkeln så räknas det som en träff, strålen behöver alltså inte skära cirkeln i två punkter, så det är ett alternativ att utföra kollisionsdetektion i två steg, om vapenstrålens startpunkt ligger i cirkeln behöver vi inte utföra test för stråle mot cirkel.

För cirkeln använd **DebugDrawer.DrawCircle**, tar **position**, **radie** och (optional) **färg**.

Exempel (pseudo-kod. Endast för referens, ni väljer själva hur det implementeras):

```
Circle::Init()
    myPosition = {randomX, randomY};
    myRadius = 0.0f;
    Circle::Update() DebugDrawer.DrawCircle(myPosition, currentRadius);
```

Tips

Håll det enkelt! Det finns **inga** krav på att kontroller mellan objekt görs på ett optimalt (eller ens bra) sätt. **Fokusera** på att lösa uppgiften med **enkla** medel.

Spelarens punkter kan testas mot skeppets linjer (*spelarens 4 punkter mot skeppets 5 linjer*).

Strålen kan kontrolleras i två steg. Först kan kontroll ske för att se om startpunkten ligger i cirkeln (*är avståndet från strålens position till cirkelns position kortare än cirkelns radie?*)

Gör inte mer än så för uppgiften, ni kommer i senare kurser lära er strategier för att sålla bort onödiga tester.

Inlämning

Kod och exe för programmet. Ta bort skräpfiler.